

Caso práctico de Servicios de IA en AWS

Pablo Rosas Plaza



Módulo/Asignatura: Programación de Inteligencia Artificial

Práctica: Caso práctico de servicios de AWS

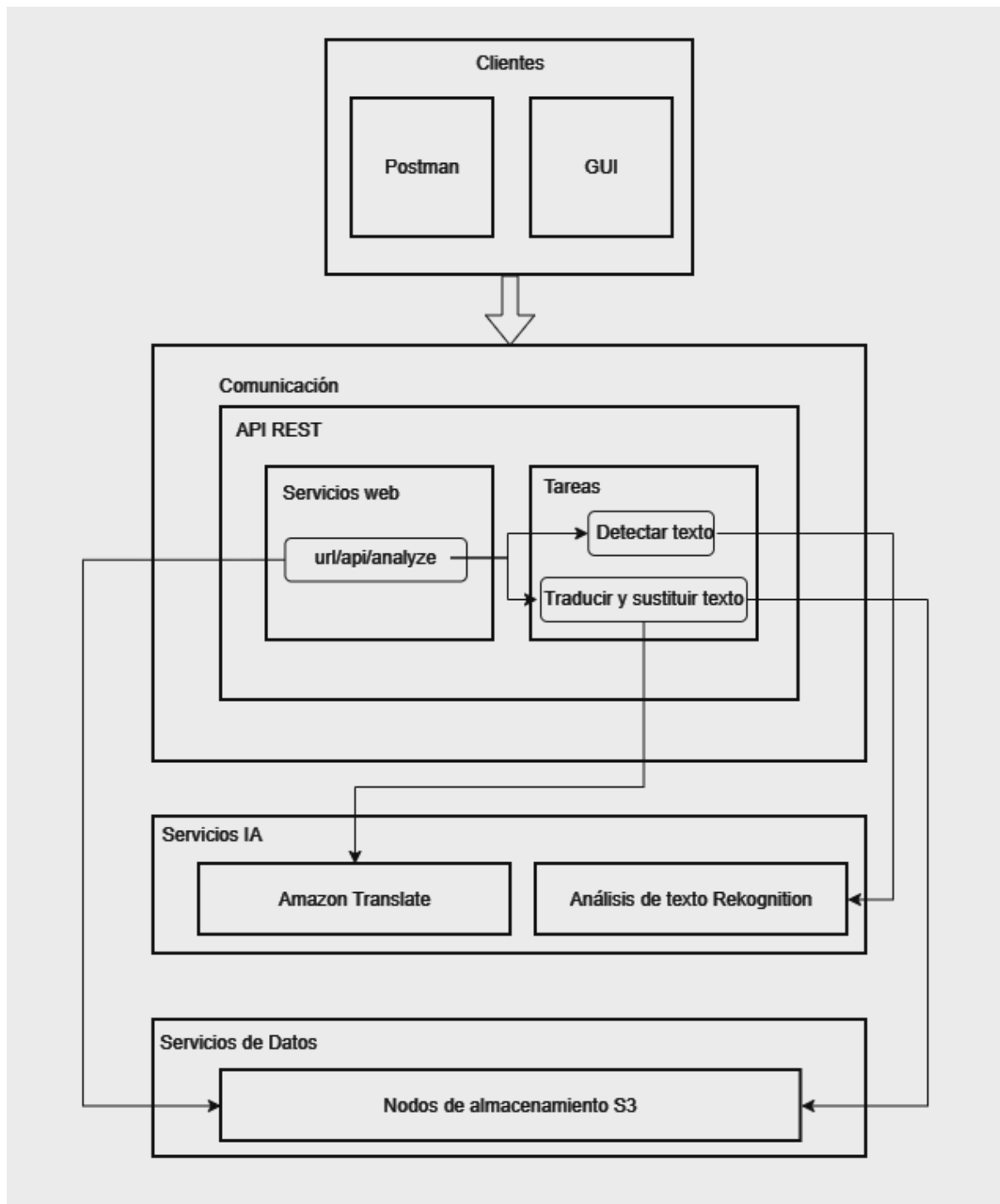
Alumno: Pablo Rosas Plaza



ÍNDICE

Arquitectura del Sistema.....	3
Flujo de Trabajo General.....	4
Módulos.....	5
Comunicación.....	5
Servicios Web.....	5
Servicios de IA.....	5
Tareas.....	5
Servicios de Almacenamiento.....	5
Servicio IA de AWS.....	6
Ejemplo de uso.....	7
Conclusión Final.....	8

Arquitectura del Sistema



Pasos:

1: Entrada

2: Proceso en Flask

3: Procesamiento

4: Salida

Flujo de Trabajo General

Los siguientes pasos describen el flujo de trabajo general:

Se lanza el Servidor Flask, el cuál se queda esperando a recibir una petición del método POST en formato json.

En el segundo paso hay dos opciones, **mandarle una petición a Flask** mediante **Postman** o hacerlo mediante la **interfaz gráfica** ejecutándola en dos consolas diferentes, en ambas se ejecutará la lógica de fondo. La interfaz gráfica te va mostrando la imagen actual, esta opción te permite además, subir imágenes al **Bucket** en la carpeta de **inputs**.

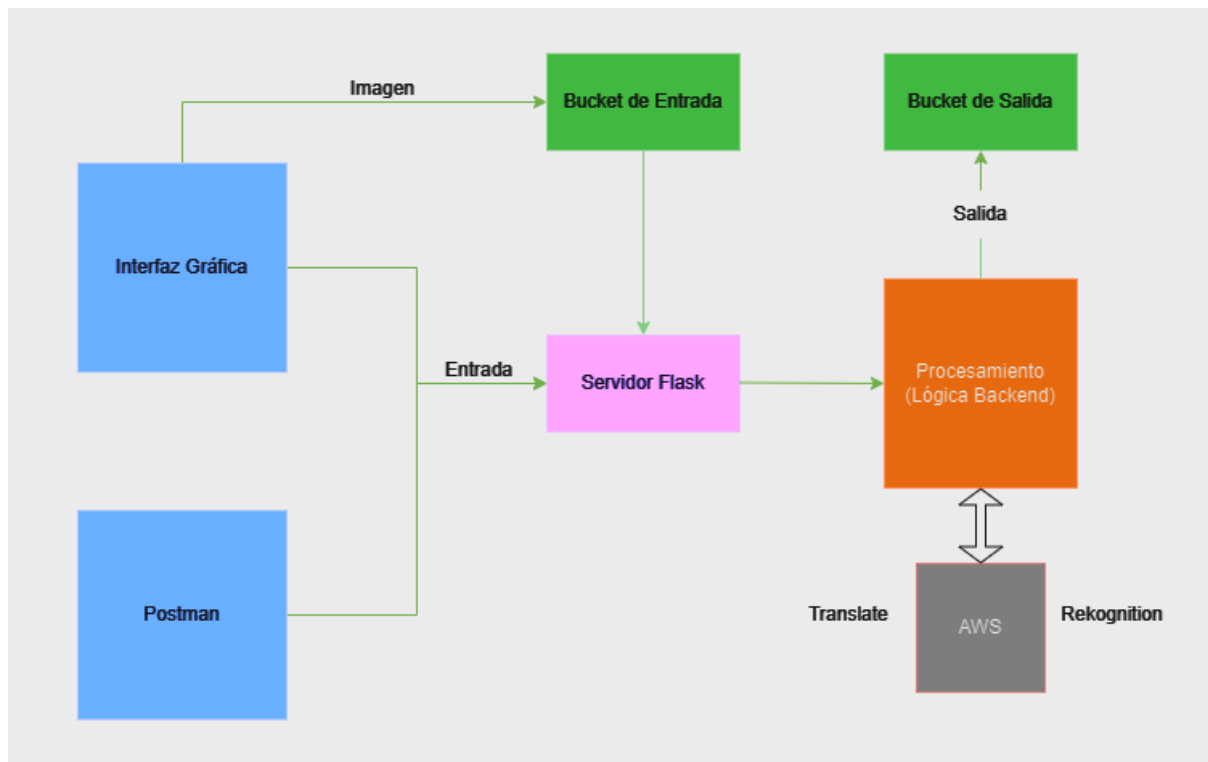
En el siguiente paso del flujo del programa, Flask recibe la petición **POST**, ya sea de Postman o de la interfaz gráfica y ejecuta la lógica del backend.

En esta lógica, primero se hace una petición a la api de **Rekognition** mediante la función `detect_text()` (de boto3), para detectar el texto de la imagen especificada del bucket. En esta respuesta en formato json, se reciben atributos tales como el texto detectado y las coordenadas de la imagen.

Después se procesa la imagen censurando aquellas partes con texto mediante la función `delete_text()`, que reemplaza los píxeles marcados como coordenadas por la respuesta de *Reckognition*, más un margen de dos píxeles para asegurarse de que no quede ningún resto del texto original.

Una vez borrado el texto original de la imagen, se llama a la api de **Translate de AWS**, con el texto detectado por *Rekognition*. *Translate* nos devuelve el texto traducido al castellano y la función `translate_and_reinsert_text()`, se encarga de elegir tamaño y tipo de letra para que ocupe el mismo espacio que el anterior texto y así colocarlo en las coordenadas donde estaba el original.

Una vez obtenida la imagen final se sube al Bucket **aws-project-bucket**, en la carpeta **output** y se actualiza la imagen de la interfaz.



Componentes clave:

- **Flask Server** para escuchar las peticiones.
- **Postman** o **Interfaz Gráfica** como opciones de entrada.
- **Bucket S3** con sus carpetas **input** y **output**.
- **AWS Rekognition** para la detección de texto.
- **AWS Translate** para la traducción de texto.
- **Procesamiento** funciones internas de la lógica del programa.

Módulos

Comunicación

Este módulo **gestiona las peticiones entre las distintas partes del sistema**. Se centra en el intercambio de datos entre cliente y servidor, así como las llamadas a APIs externas. La comunicación sigue el protocolo *HTTP/HTTPS* para garantizar la transmisión segura de datos hacia y desde AWS.

Tecnologías utilizadas:

- **Flask**: Permite recibir y procesar las solicitudes POST enviadas por Postman o la interfaz gráfica.
- **Requests**: Se utiliza para enviar datos a los servicios web de AWS, como S3, *Rekognition* y *Translate*.

Servicios Web y Servicios IA

Este módulo está compuesto por las **APIs utilizadas para** interactuar con los **servicios de AWS** y procesar las imágenes. Se realizan solicitudes específicas a cada API usando la librería **boto3**, que permite interactuar con AWS desde Python.

Servicios utilizados:

- **AWS S3:** Almacena las imágenes originales en la carpeta **input** y las procesadas en la carpeta **output**.
- **AWS Rekognition:** Detecta texto en las imágenes subidas en el bucket S3.
 - *Salida:* Un JSON que incluye el texto detectado y sus coordenadas.
- **AWS Translate:** Traduce el texto detectado al idioma objetivo.
 - *Salida:* Texto traducido en formato JSON.

Tareas

Este módulo agrupa las **funciones que permiten integrar las salidas de los servicios de IA con el procesamiento de la imagen**.

Las principales funciones que usa mi programa son **delete_text()**, que borra el texto detectado en la imagen utilizando las coordenadas proporcionadas por **Rekognition** y **translate_and_reinsert_text()** encargada de reinserta el texto traducido en las posiciones originales, ajustando tamaño y fuente según el contexto de la imagen.

Otras funciones a destacar son **detect_text()** que hace la petición a **Rekognition** de detectar el texto de la imagen, y **filter_overlapping_detections()** que contiene la lógica para decidir que boundingbox se superpone, ya que **Rekognition** devuelve el texto reconocido como frase y como palabras, es decir lo duplica, por eso hay que limpiarlo para eliminar duplicados.

Librerías Usadas:

- Librería de manipulación de imágenes **Pillow (PIL)**, para editar las imágenes píxel a píxel.
- Librería de conexión con **AWS boto3**.

Servicios de Almacenamiento

Este módulo administra el almacenamiento de las imágenes en el sistema. Incluye la gestión de los buckets en AWS S3. El almacenamiento es clave para la integración entre los servicios. Cada imagen se sube y se descarga del bucket utilizando URLs generadas dinámicamente.

Almacenamiento Usado:

- **AWS S3:**
 - Carpeta **input**: Almacena las imágenes originales cargadas desde la interfaz gráfica.
 - Carpeta **output**: Guarda las imágenes procesadas con el texto traducido.

Servicio IA de AWS

AWS Rekognition

AWS Rekognition se utiliza para detectar elementos dentro de imágenes almacenadas en un bucket de *S3*, pueden ser tanto texto como imágenes concretas como objetos, personas, etc.

Flask realiza una **solicitud** al servicio *Rekognition* mediante la función **detect_text()** del *SDK* de *AWS* (*Boto3* en Python). En la **solicitud**, se especifica el **nombre del bucket** y la **ubicación de la imagen**.

Rekognition procesa la imagen y **responde** con un JSON con la información del texto detectado, a modo de lista.

TextDetections:

- **DetectedText**: Texto detectado en la imagen.
- **Type**: Tipo de texto detectado (**LINE** o **WORD**).
- **Id**: Identificador único del elemento detectado.
- **Confidence**: Confianza del modelo en la precisión de la detección (en porcentaje).
- **Geometry**: Coordenadas del área donde se detectó el texto.
 - **BoundingBox**: Caja delimitadora que encierra el texto.
 - **Width** y **Height**: Dimensiones de la caja en proporción al tamaño de la imagen.
 - **Left** y **Top**: Coordenadas de la esquina superior izquierda de la caja (valores entre 0 y 1).
 - **Polygon**: Coordenadas de los vértices del área donde se detectó el texto (usado para casos más complejos).

AWS Translate

AWS Translate se utiliza para traducir texto, en este caso el detectado por *Rekognition* al idioma deseado (español). Flask envía una solicitud al servicio **Translate** mediante la función **translate_text()** del SDK de *AWS*.

En la **solicitud**, se especifica:

- El texto a traducir.
- El idioma de origen (opcional, puede ser detectado automáticamente).
- El idioma de destino.

Mientras que en la respuesta se devuelve:

- **TranslatedText**: Texto traducido.
- **SourceLanguageCode**: Idioma de origen detectado.
- **TargetLanguageCode**: Idioma de destino.

Ejemplo de uso

Antes de nada, asegurarse de tener las dependencias instaladas, basta con ejecutar el comando pip install sobre el archivo requirements.txt

```
pip install -r requirements.txt
```

Empezamos ejecutando el servidor Flask en la terminal:

```
Microsoft Windows [Versión 10.0.19045.5247]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\asrau\Desktop\aws-proyect>python ./application.py
* Serving Flask app 'application'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 119-265-344
```

Después lanzamos la interfaz gráfica (opcional, se puede hacer desde Postman)

```
C:\Windows\System32\cmd.exe - python ./front.py

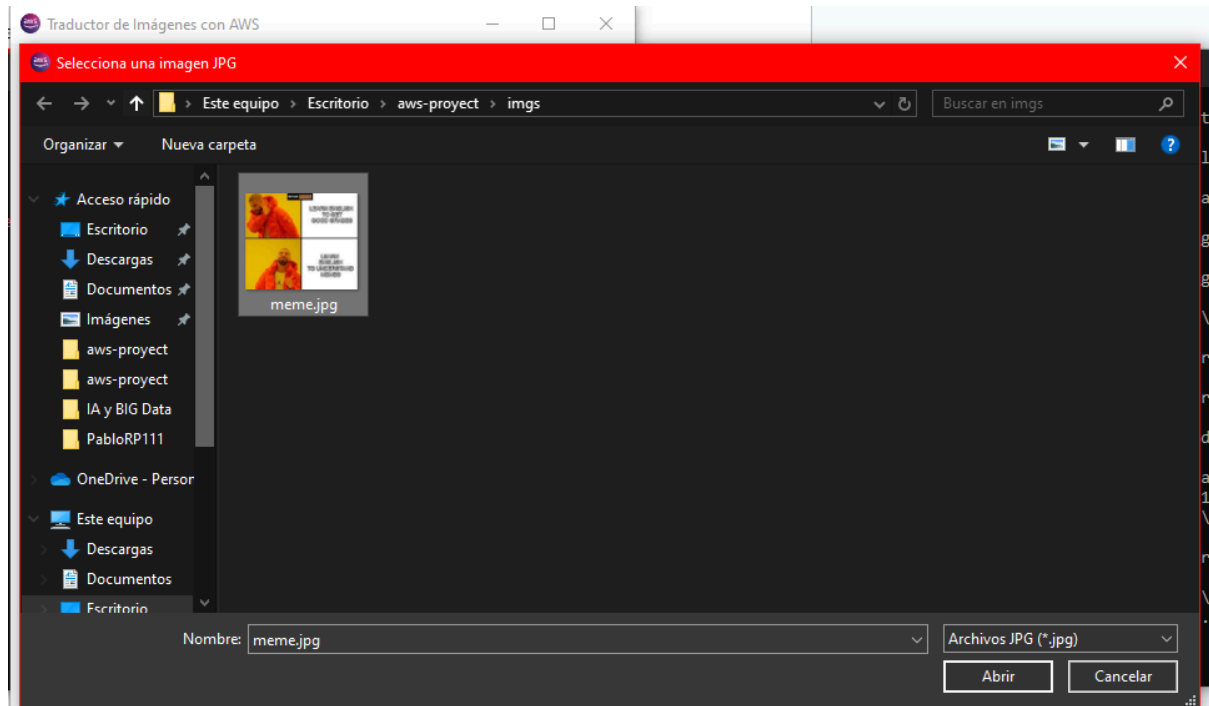
Microsoft Windows [Versión 10.0.19045.5247]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\asrau\Desktop\aws-proyect>python ./front.py
```

Seleccionamos una imagen para subirla al bucket (si estamos en postman ponemos una ya subida)



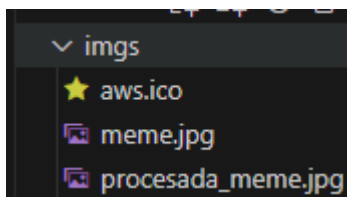
Hay una imagen de prueba en la carpeta, imgs, también puedes probar con otras.



Le damos a procesar (o botón para enviar metodo POST en Postman)



Y Voilà!! ya tenemos el texto de la imagen traducida y está almacenada en la carpeta imgs:



Conclusión Final

Este proyecto ha resultado ser una excelente práctica del uso de AWS y sus servicios, integrándose con Python y Flask. Me ha servido para reforzar conocimientos y aprender nuevos. En conclusión me ha parecido un proyecto muy completo.